

第5章: Supabaseのセットアップ

アプリで登録した本のデータは、どこかに保存しておく必要があります。この章では、データの保管庫 = バックエンド／データベースを用意します。まず「どの保存方法を選ぶべきか」を比較したうえで、本書が採用する Supabase をセットアップし、アプリから接続できるようにします。

1. そもそも「データを保存する」とは

第1章で作ったアプリは、一度閉じると入力した内容が消えてしまいます。本のリストをずっと残しておくには、データを「永続化（えいぞくか）」する場所が必要です。

「永続化（persistence）」とは？「アプリを閉じて、スマホを再起動しても、データが消えずに残ること」。逆に、アプリの実行中だけ覚えている（閉じると消える）状態を「一時的（揮発性）」と呼びます。state は一時的、データベースは永続的です。

データの保存先には、大きく分けて2つの場所があります。

場所	特徴	例
端末内（ローカル）	そのスマホの中だけに保存。ネット不要だが、別の端末とは共有できない	SQLite、AsyncStorage
クラウド（サーバー）	インターネットの向こうのサーバーに保存。複数端末で同じデータを共有できる	Supabase、Firebase

本書では、複数端末で同じ本棚を見られるよう、クラウド型の Supabase を使います。

2. データ保存方法の選択肢を比較する

「どんな場合にどれを選ぶべきか」を、第0章よりさらに詳しく解説します。アプリの目的によって最適な選択は変わります。

2.1 主要な選択肢の比較表

選択肢	種類	置き場所	複数端末 同期	オフ ライン	SQL	向いているアプリ
Supabase（本書採用）	RDB （PostgreSQL）	クラウド	◎	△	○	SNS、共有する本棚、複数 端末で使うもの
Firebase （Firestore）	NoSQL	クラウド	◎	◎	×	チャット、リアルタイム共同編 集
expo-sqlite	RDB（SQLite）	端末内	×	◎	○	完全オフラインのメモ・家計 簿
AsyncStorage	キー・バリュー	端末内	×	◎	×	設定値・小さなデータの保存
WatermelonDB	RDB（SQLiteベ ース）	端末内＋ 同期	○	◎	△	大量データ＋オフライン重視

2.2 それぞれの解説

◆ Supabase（本書の採用）

Supabase（スーパベース）は、PostgreSQL（ポストグレスキューエル、業界標準のRDB）をベースにした「BaaS（Backend as a Service）」です。

「BaaS」とは？ Backend as a Service の略。「バックエンド（サーバー側の機能）を、自分で作らずサービスとして借りる」という意味。データベース・API・認証などを、設定だけで使えます。

- **長所:** SQL（業界標準のデータ操作言語）が使える／複数端末で同期できる／無料枠が十分／本書のWeb版（Next.js）とまったく同じバックエンドを共用できる／オープンソース
- **短所:** 完全オフライン動作には工夫が必要

◆ Firebase（Firestore）

Googleが提供するBaaS。データベースは **Firestore** という **NoSQL**（表形式でない）型。

- **長所:** リアルタイム同期やオフライン対応が得意／Googleサービスと連携しやすい
- **短所:** SQLが使えず独自の操作方法を覚える必要がある／他サービスへの移行がしにくい

◆ expo-sqlite（端末内DB）

スマホの中に **SQLite**（軽量のRDB）を持つ方式。

- **長所:** ネット完全不要／高速／サーバー費用ゼロ
- **短所:** そのスマホの中だけ。機種変更やPC・別端末とは共有できない

◆ AsyncStorage

「キーと値」だけの最もシンプルな端末内保存。設定値などの小さなデータ向け。

どんな時にどれを選ぶ？（まとめ） - 複数端末で同期したい／Webアプリとデータを共有したい → Supabase（本書） - チャットなどリアルタイム性が最優先 → Firebase - ネット不要の完全オフライン・自分専用アプリ → expo-sqlite - テーマ設定やログイン状態など小さな値だけ → AsyncStorage（他のDBと併用も多い）

3. Supabaseのアカウント作成とプロジェクト作成

Web版チュートリアルを既にやった方へ: 同じSupabaseプロジェクトを使い回せます。その場合は「3.4 既存テーブルの確認」へ進み、**books** テーブルがあれば新規作成は不要です。

3.1 アカウント作成

1. ブラウザで <https://supabase.com/> を開きます。
2. 「Start your project」または「Sign In」をクリック。
3. GitHubアカウントでサインアップするのが簡単です（第1章でGitHubを使う前提）。メールアドレスでの登録も可能です。

3.2 新しいプロジェクトを作る

1. ダッシュボード（管理画面）で「New project」をクリック。
2. 以下を入力します。

項目	入力内容
Name	プロジェクト名（例: books-app ）。自分が分かる名前でもOK
Database Password	データベースのパスワード。 必ず控えておく （後で必要）
Region	サーバーの場所。日本なら「 Northeast Asia (Tokyo) 」が高速

3. 「Create new project」を押すと、数分でデータベースが用意されます。

「Region（リージョン）」を東京にする理由: サーバーが物理的に近いほど、データのやり取りが速くなります。日本のユーザー向けアプリなら東京を選ぶと体感速度が上がります。

3.3 テーブルを作る — **books** テーブル

データベースの中に「本の情報を入れる表（テーブル）」を作ります。Supabaseの **SQL Editor**（SQLを書いて実行する画面）を使うのが確実です。

1. Supabaseの左メニューから「**SQL Editor**」を開きます。
2. 次のSQLを貼り付けて、右下の「**Run**」を押します。

```
-- create table : 新しいテーブル（表）を作るSQL命令
-- books       : テーブル名（本のデータを入れる表）
create table books (
  -- id : 各行を区別する識別番号。uuid型で、自動生成される
  id uuid primary key default gen_random_uuid(),
  -- primary key : 「主キー」。各行を一意に識別する特別な列
  -- default gen_random_uuid() : 値を省略したら自動でランダムなIDを振る

  title text not null,          -- title : 本のタイトル。text型（文字列） / not null = 空は許さない（必須）
  author text not null,         -- author : 著者名。必須
  status text not null default '未読', -- status : 読書状態。省略時は '未読' になる (default)
  memo text,                   -- memo : メモ。not nullが無いので空でもOK（省略可能）

  -- created_at : 登録日時。timestamp型で、登録時の時刻が自動で入る
  created_at timestamp with time zone default now()
  -- default now() : 省略時は「今の時刻」を自動で入れる
);
```

SQLの各行の意味: - **create table books (...)** ... 「booksという名前の表を作る」命令。 - 各行は「**列名 + 型 + 制約**」の形。**title text not null** は「titleという列、文字列型、空は不可」。 - **primary key**（主キー）... 各行を見分けるための特別な列。本でいう「管理番号」。 - **not null** ... 「この列は必ず値を入れる」というルール（制約）。 - **default 値** ... 「省略したらこの値を入れる」初期値の指定。

3.4 （任意）テストデータを入れる

動作確認用に、最初から数冊入れておくと便利です。同じくSQL Editorで実行します。

```
-- insert into : テーブルにデータ（行）を追加するSQL命令
-- books (title, author, status) : どの列に値を入れるか指定
-- values (...) : 入れる値。1行が1冊分。複数行はカンマで区切る
insert into books (title, author, status) values
('リーダブルコード', 'Dustin Boswell', '読了'),
('達人プログラマー', 'David Thomas', '読書中'),
('プロを目指す人のためのTypeScript入門', '鈴木 僚太', '未読');
```

3.5 行レベルセキュリティ（RLS）の設定

Supabaseは安全のため、初期状態では外部からデータを読み書きできないようロックされています。学習用に、まずは「誰でも読み書きできる」設定にします。

```
-- alter table ... enable row level security : このテーブルに「行レベルセキュリティ」を有効化
alter table books enable row level security;

-- create policy : アクセスを許可する「ポリシー（規則）」を作る
-- 学習用に「全員に全操作を許可」する規則を作成（本番アプリでは後で厳しくする）
create policy "Enable all access for books"    -- ポリシーの名前（自由）
  on books                                     -- 対象テーブル
  for all                                       -- for all : 全操作（読み・書き・更新・削除）
  を対象に
  using (true)                                -- using (true) : 読み取り条件＝常に許可
  with check (true);                          -- with check (true) : 書き込み条件＝常に許可
可
```

「RLS（Row Level Security）」とは？「行（Row）ごとに、誰がアクセスできるかを制御するセキュリティ」のこと。本来は「自分のデータは自分しか見られない」のように細かく制御します。本書は学習用なので一旦「全員OK」にしますが、実際に公開するアプリでは必ず適切に制限してください（第10章で再度触れます）。

4. アプリからSupabaseに接続する準備

4.1 接続情報（URLとキー）を取得する

アプリからSupabaseにつなぐには、2つの情報が必要です。

- Supabaseダッシュボードの左メニュー「Project Settings（歯車アイコン）」→「API」を開きます。
- 次の2つを控えます。

項目	何か
Project URL	あなたのデータベースの住所（ https://xxxx.supabase.co の形）
anon public key	アプリから接続するための公開鍵（長い文字列）

「anon public key」は公開してOK？ これは「匿名（anonymous）ユーザー用の公開鍵」で、アプリに埋め込む前提のキーです。データの保護は前述のRLS（行レベルセキュリティ）で行います。一方、**service_role** という別のキーは絶対に公開してはいけない管理者キーなので、アプリには入れないでください。

4.2 必要なライブラリをインストールする

第1章で作った `my-books-app` フォルダで、ターミナルから次を実行します。

```
npx expo install @supabase/supabase-js @react-native-async-storage/async-storage react-native-url-polyfill
# npx expo install : Expo環境に合ったバージョンで部品をインストールするコマンド (npm installのExpo版)
# @supabase/supabase-js : SupabaseをJS/TSから操作する公式ライブラリ
# @react-native-async-storage/async-storage : ログイン状態などを端末に保存する補助 (Supabaseが内部で使う)
# react-native-url-polyfill : React NativeでURL機能を補う部品 (Supabaseの動作に必要)
```

なぜ `npm install` ではなく `npx expo install` ? `npx expo install` は「今のExpoのバージョンと相性の良い版」を自動で選んでくれます。React Native関連の部品はバージョンの相性が重要なので、Expoプロジェクトでは原則こちらを使います。

4.3 環境変数を設定する（接続情報を安全に持つ）

接続情報をコードに直接書かず、**環境変数**（かんきょうへんすう）という外部ファイルに置きます。プロジェクト直下に `.env` というファイルを新規作成し、次を書きます。

```
# .env ファイル（プロジェクトの一番上の階層に置く）
# EXPO_PUBLIC_ で始まる名前にすると、Expoアプリ内から読み取れる決まり
EXPO_PUBLIC_SUPABASE_URL=https://あなたのプロジェクト.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=あなたのanon_publicキー
# = の右側に、4.1で控えた値を貼り付ける（クォートで囲む必要はない）
```

「環境変数」と `.env` ファイルとは？「アプリの外側に置く設定値」のことです。接続情報やパスワードをコードに直書きすると、GitHubに公開したとき他人に見られて危険です。`.env` という別ファイルに分け、これを**Gitの管理から外す**ことで安全を保ちます。

EXPO_PUBLIC_ という接頭辞の意味: Expoでは、`EXPO_PUBLIC_` で始まる環境変数だけがアプリのコードから読めます。逆にこの接頭辞が無い変数はアプリに含まれません。なお **PUBLIC**（公開）の名の通り、ここに入れた値は最終的にアプリに埋め込まれるので、`service_role` のような秘密キーは入れないこと。

`.env` をGit管理から外すため、`.gitignore` というファイル（無ければ作成）に次の1行を追加します。

```
# .gitignore ファイル（Gitに無視させたいファイルを列挙する）
.env
# ↑ この行で「.envファイルはGitで記録しない（=GitHubにアップしない）」という指定になる
```

.gitignore とは？「Gitに記録（追跡）させたくないファイルの一覧」を書くファイルです。**.env**（秘密情報）や **node_modules**（巨大な部品本体）などを書いておくと、GitHubにアップロードされず安全 & 軽量になります。Expoのテンプレートには最初から **.gitignore** があり、**node_modules** などは既にかかれています。

4.4 Supabaseクライアントを作る

アプリ内で何度も使う「Supabaseへの接続オブジェクト」を1か所にまとめます。プロジェクト内に **lib** フォルダを作り、その中に **supabase.ts** を新規作成します。

```
// lib/supabase.ts - Supabaseへの接続を作る共通ファイル

import "react-native-url-polyfill/auto"; // RNでURL機能を有効化（おまじない。先頭で読み込む）
import AsyncStorage from "@react-native-async-storage/async-storage"; // 端末保存の部品
import { createClient } from "@supabase/supabase-js"; // Supabaseクライアントを作る関数を借りる

// process.env.XXX : .envファイルに書いた環境変数を読み取る書き方
const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!; // 末尾の ! は「nullではないと保証」する印
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!;

// createClient(URL, キー, 設定) : Supabaseへの接続オブジェクトを作る
export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: AsyncStorage, // ログイン状態を端末(AsyncStorage)に保存する設定
    autoRefreshToken: true, // 認証トークンを自動更新する
    persistSession: true, // セッション（ログイン状態）を保持する
    detectSessionInUrl: false, // URLからのセッション検出はモバイルでは不要なのでオフ
  },
});
// export しているので、他のファイルから import { supabase } from "../lib/supabase" で使える
```

！（エクスクラメーション）の意味: **process.env.EXPO_PUBLIC_SUPABASE_URL!** の末尾の **!** は、TypeScriptに「この値はnull/undefinedではないと約束する」と伝える印です。環境変数は型の上では「無いかもしれない」扱いなので、**!** で「ちゃんと設定してあるから大丈夫」と明示しています。

4.5 接続テスト

きちんとつながるか確認しましょう。任意の画面ファイル（例: **app/(tabs)/index.tsx**）に、一時的に次を足して確認します。

```
import { useEffect } from "react";
import { supabase } from "../../lib/supabase"; // 作った接続オブジェクトを借りる（パスは場所に応じて調整）

export default function HomeScreen() {
  // useEffect : 画面表示時に1回だけ実行（第3章参照）
  useEffect(() => {
    // 即時実行する非同期関数（async/awaitを使うため）。第2章のasync/await参照
    const test = async () => {
      // supabase.from("books").select("*") : booksテーブルから全列(*)を取得する
      // await : 結果が返るまで待つ / { data, error } : 結果を分割代入で受け取る
      const { data, error } = await supabase.from("books").select("*");
      if (error) {
        console.log("接続エラー:", error.message); // 失敗時はエラー内容を表示
      } else {
        console.log("取得した本:", data); // 成功時はデータを表示
      }
    };
    test();
  }, []);

  // （画面の見た目は省略。ターミナルのログで結果を確認する）
}
```

確認方法: アプリを起動（`npx expo start`）してこの画面を開き、**ターミナル**を見ます。「取得した本: [...]」と3冊分のデータ（3.4でテストデータを入れた場合）が表示されれば、接続成功です！「接続エラー」が出たら、`.env`の値が正しいか、RLSのポリシー（3.5）を設定したかを確認してください。

確認できたら消す: このテストコードは確認用です。動作確認できたら消して、次章からの本実装に進みます。

5. この章のまとめ

- データを永続化するには**端末内**か**クラウド**の保存先が必要。本書は複数端末同期できる**クラウド型**のSupabaseを採用
- 保存方法の選択肢（Supabase / Firebase / SQLite / AsyncStorage）を比較し、目的別の選び方を理解した
- Supabaseでプロジェクトと **books** テーブルをSQLで作成し、学習用にRLS（行レベルセキュリティ）を設定した
- 接続情報を `.env`（環境変数）に安全に保管し、`.gitignore` でGit管理から除外した
- `lib/supabase.ts` にSupabaseクライアントを作り、接続テストに成功した

次の章へ: データの保管庫が整いました。第6章では、アプリの画面構成（ナビゲーション）を本格的に組み立て、第7・8章でいよいよCRUD（一覧・作成・編集・削除）を実装します。